ASP.NET Core Identity

Lernunterlage: Benutzer, Passwörter und Rollen nicht selbst bauen

1 Warum Identity?

Benutzerverwaltung ist voller Fallen: Passwörter richtig hashen, doppelte E-Mail-Adressen, Rollen, Sperre nach Fehlversuchen, Groß- und Kleinschreibung. **Sicherheitscode schreibt man nicht selbst**, man nimmt das geprüfte Framework. ASP.NET Core Identity ist genau das.

	Vorher (selbst gebaut)	**Jetzt (Identity)**
Benutzerklasse	Eigene Klasse mit Email, PasswordHash, Role	**User : IdentityUser<Guid>**, Felder kommen von Identity
Passwort speichern	PasswordHasher von Hand aufrufen, Hash selbst eintragen	**UserManager.CreateAsync(user, password)**
Passwortregeln	Eigene Prüfung im Endpunkt	Identity-Optionen, Fehler als IdentityResult
Rolle	Textfeld im Benutzer	Eigene Tabellen (AspNetRoles, AspNetUserRoles)
Doppelte E-Mail	Eigene Abfrage	RequireUniqueEmail, Identity prüft
Fehlversuche	keine Sperre	Lockout: 5 Fehler, 5 Minuten gesperrt
DbContext	DbContext	**IdentityDbContext<User, IdentityRole<Guid>, Guid>**

Merksatz: Identity verwaltet **Benutzer** (Wer bist du?). Das **JWT** ist nur der Ausweis, den Home Control danach ausstellt. Beides arbeitet zusammen, ersetzt sich aber nicht.

2 Begriffe

Begriff	**Bedeutung**
IdentityUser<Guid>	Fertige Benutzerklasse mit Id (Guid), UserName, Email, PasswordHash, SecurityStamp, Lockout-Feldern, PhoneNumber ...
IdentityRole<Guid>	Fertige Rollenklasse (Name, Id). Bei uns: Admin und User.
IdentityDbContext	DbContext, der alle Identity-Tabellen mitbringt. Unser ApplicationDbContext erbt davon.
UserManager<User>	Die Schaltzentrale: Benutzer anlegen, suchen, Passwort prüfen, Rollen zuweisen, Fehlversuche zählen.
RoleManager	Rollen anlegen und prüfen (RoleExistsAsync, CreateAsync).
IdentityResult	Ergebnis einer Identity-Aktion: Succeeded oder eine Liste von Errors (Code und Beschreibung).
AddIdentityCore	Registriert die Benutzerverwaltung ohne Cookie-Anmeldung und ohne fertige Oberfläche. Wir melden mit JWT an.
Normalisierung	Identity speichert zusätzlich NormalizedEmail und NormalizedUserName in Großbuchstaben, damit die Suche Groß-/Kleinschreibung ignoriert.

Begriff	Bedeutung
SecurityStamp	Zufallswert pro Benutzer. Ändert sich bei sicherheitsrelevanten Änderungen, z. B. neuem Passwort.
Lockout	Zeitweise Sperre des Kontos nach zu vielen falschen Passwörtern.
RBAC	Role-Based Access Control: Rechte hängen an Rollen (Admin, User), nicht an einzelnen Personen.

3 Der Benutzer

Die Klasse User erbt von IdentityUser<Guid>. Dadurch gibt es Id, UserName, Email und PasswordHash schon. **Felder nie doppelt definieren**, sonst hat man zwei E-Mail-Spalten. Hinzu kommen nur die Dinge, die Identity nicht kennt:

```
public class User : IdentityUser<Guid>
{
    public DateTimeOffset CreatedAtUtc { get; set; }
    public List<RefreshToken> RefreshTokens { get; set; } = [];
}
```

Die Rolle steht **nicht mehr** im Benutzer. Sie liegt in der Tabelle AspNetUserRoles und wird mit **UserManager.AddToRoleAsync** gesetzt und mit **GetRolesAsync** gelesen.

4 Datenbank: IdentityDbContext und Tabellen

```
public class ApplicationDbContext
    : IdentityDbContext<User, IdentityRole<Guid>, Guid>
// in OnModelCreating zuerst:  base.OnModelCreating(modelBuilder);
```

Tabelle	Inhalt
AspNetUsers	Benutzer (E-Mail, PasswordHash, SecurityStamp, Lockout, CreatedAtUtc)
AspNetRoles	Rollen Admin und User
AspNetUserRoles	Wer hat welche Rolle (Verbindungstabelle)
AspNetUserClaims, AspNetRoleClaims	Zusätzliche Angaben zu Benutzer oder Rolle (bei uns ungenutzt)
AspNetUserLogins, AspNetUserTokens	Anmeldung über Google & Co. und Bestätigungstokens (bei uns ungenutzt)
RefreshTokens (eigene)	Unsere Refresh-Tokens, zeigen auf AspNetUsers
Rooms, Devices, DeviceEvents, BlinkCredentials (eigene)	Bleiben unverändert

Das Wichtigste: Alle eigenen DbSets bleiben, und Tuya- und Blink-Daten gehen nicht verloren.

5 Registrierung

Schritt	Wer	Was passiert
1	Endpunkt	POST /api/auth/register: E-Mail und Passwort grob prüfen (leer? zu lang?)
2	AuthService	Erster Benutzer? Dann darf jeder, er wird Admin. Sonst nur ein angemeldeter Admin

Schritt	Wer	Was passiert
3	AuthService	User-Objekt bauen (UserName = E-Mail, klein geschrieben)
4	UserManager	**CreateAsync(user, password)**: Regeln prüfen, Salt erzeugen, PBKDF2-Hash bilden, speichern
5	AuthService	Rolle anlegen, falls es sie noch nicht gibt, dann **AddToRoleAsync**
6	Endpunkt	201 Created, oder bei Fehlern 400 mit ValidationProblem

IdentityResult in ValidationProblem übersetzen

CreateAsync wirft bei Fehlern **keine Ausnahme**, sondern liefert ein IdentityResult. Bei Succeeded = false stehen in **result.Errors** Code und Text, zum Beispiel PasswordTooShort oder DuplicateEmail. Der Endpunkt macht daraus die gewohnte ProblemDetails-Antwort (HTTP 400), damit alle Fehler gleich aussehen. Die Texte kommen über den GermanIdentityErrorDescriber auf Deutsch.

```
var created = await userManager.CreateAsync(user, password);
if (!created.Succeeded)
    return Results.ValidationProblem(ToValidationErrors(created.Errors));
```

6 Passwort-Hashing

- Das Passwort wird **nie gespeichert**, nur ein Hash.
- Identity nutzt **PBKDF2** mit zufälligem **Salt** und vielen Durchläufen. Das ist absichtlich langsam.
- Im PasswordHash-Feld stehen Verfahren, Salt und Hash in einem Text.
- Beim Login hasht Identity das eingegebene Passwort mit demselben Salt und vergleicht. Zurückrechnen geht nicht.
- Ist ein Hash veraltet (weniger Durchläufe), hasht Identity beim nächsten erfolgreichen Login automatisch neu.

> **Merksatz:** Nie selbst hashen, nie SHA-256 für Passwörter. SHA-256 ist schnell, und schnell ist bei Passwörtern schlecht. (Für unsere Refresh-Tokens ist SHA-256 dagegen richtig: Das sind lange Zufallswerte, keine Passwörter.)

7 Rollen (RBAC)

Rolle	Darf
Admin	Räume, Geräte und Ereignisse anlegen, ändern, löschen; neue Benutzer anlegen
User	Alles ansehen (lesen)

- Rollen entstehen beim ersten Gebrauch (RoleManager.CreateAsync).
- Die Rolle kommt als Claim **role** ins JWT. Die Regel AdminOnly (RequireAuthorization) prüft sie.
- **401** = nicht angemeldet. **403** = angemeldet, aber keine Berechtigung.

8 Sperre nach Fehlversuchen (Lockout)

Ohne Sperre könnte ein Angreifer beliebig viele Passwörter durchprobieren (Brute Force). Mit Identity:

1. Benutzer über FindByEmailAsync suchen.
2. IsLockedOutAsync: schon gesperrt? Dann sofort Antwort 429, ohne das Passwort zu prüfen.
3. CheckPasswordAsync: stimmt das Passwort?
4. Falsch: AccessFailedAsync zählt hoch. Beim fünften Fehler sperrt Identity für 5 Minuten.
5. Richtig: ResetAccessFailedCountAsync setzt den Zähler zurück, dann werden die Tokens ausgestellt.

9 Identity und JWT zusammen

Frage	Antwort in Home Control
Wer verwaltet Benutzer und Passwörter?	Identity (UserManager)
Woher weiß die API bei jeder Anfrage, wer ruft?	Aus dem JWT (Access Token, 15 Minuten)
Wie kommt man an ein neues JWT, ohne Passwort?	Refresh-Token im HTTP-only Cookie, mit Rotation
Wo steht, was das JWT enthält?	sub (Id), email, role. Kein Passwort, keine Tuya- oder Blink-Daten
Warum nicht Cookie-Anmeldung von Identity?	Wir haben ein getrenntes React-Frontend und die API wird auch vom Agenten benutzt, das passt besser zu Tokens

Nicht verwechseln: Das **Blink-Refresh-Token** gehört zum Blink-Konto (wird verschlüsselt gespeichert). Das **Home-Control-Refresh-Token** gehört zu unseren Benutzern (nur der Hash wird gespeichert).

10 Code-Landkarte

Datei	Inhalt
Models/User.cs	User : IdentityUser<Guid>, Konstanten Roles.Admin und Roles.User
Data/ApplicationDbContext.cs	IdentityDbContext, base.OnModelCreating, eigene Tabellen
Auth/IdentityRegistration.cs	AddIdentityCore, AddRoles, AddEntityFrameworkStores, Regeln, deutsche Fehlertexte
Auth/AuthService.cs	RegisterAsync, LoginAsync, RefreshAsync, LogoutAsync
Auth/AuthUser.cs	Kleines Ergebnisobjekt: Id, E-Mail, Rolle
Auth/TokenService.cs	JWT erzeugen, Refresh-Token erzeugen und hashen
Endpoints/AuthEndpoints.cs	HTTP-Endpunkte, Cookie, IdentityResult in ValidationProblem
Program.cs	builder.Services.AddHomeControlIdentity()
Tests: AuthServiceTests, TokenServiceTests	Register, Login, Sperre, Refresh, Rotation, JWT

11 Die Umstellung und ihre Grenzen

- Die Migration muss auf dem eigenen Rechner erzeugt werden: **dotnet ef migrations add IdentityUmgestellt**. Alte Migrationen werden nie verändert.
- Vorher sichert und leert ./identity-vorbereiten.sh nur die alten Benutzer und Refresh-Tokens. Alte Benutzer werden nicht übernommen, das Konto wird neu angelegt (erster Benutzer =

Admin).

- Der Code wurde ohne .NET SDK geschrieben und ist erst nach **dotnet build** und **dotnet test** als funktionierend zu betrachten.
- Noch nicht dabei: E-Mail-Bestätigung, Passwort-Zurücksetzen, Zwei-Faktor-Anmeldung.
- Eine Rolle pro Benutzer (Identity kann mehrere).
- Identity misst die Sperrzeit mit der echten Uhr, der Ablauf ist deshalb nicht im Unit-Test geprüft.

12 Kurzantworten für Rückfragen

Frage	Antwort in einem Satz
Warum Identity statt selbst bauen?	Sicherheitscode schreibt man nicht selbst. Identity ist geprüft und bringt Hashing, Regeln, Rollen und Sperre mit.
Was macht UserManager?	Er legt Benutzer an, prüft Passwörter, setzt Rollen und zählt Fehlversuche.
Was ist ein IdentityResult?	Das Ergebnis einer Identity-Aktion: Erfolg oder eine Liste von Fehlern.
Warum erbt der DbContext von IdentityDbContext?	Damit die Identity-Tabellen in unserer Datenbank mit angelegt werden.
Was ist ein Salt?	Ein Zufallswert pro Passwort, damit gleiche Passwörter verschiedene Hashes ergeben.
Warum PBKDF2 und nicht SHA-256 für Passwörter?	PBKDF2 ist absichtlich langsam, SHA-256 ist schnell und damit leichter zu erraten.
Was ist der Unterschied zwischen 401 und 403?	401: nicht angemeldet. 403: angemeldet, aber nicht erlaubt.
Wofür ist das JWT, wenn Identity die Benutzer hat?	Identity kennt die Benutzer, das JWT ist der Ausweis für jede weitere Anfrage.
Was passiert bei fünf falschen Passwörtern?	Das Konto ist 5 Minuten gesperrt, die Antwort ist 429.
Was ist mit Tuya und Blink?	Nicht betroffen: eigene Tabellen und Klassen.

13 Merksätze

- Passwörter **nie selbst hashen**: UserManager.CreateAsync(user, password).
- Identity = **Benutzerverwaltung**, JWT = **Ausweis**.
- User erbt von IdentityUser<Guid>, Felder **nicht doppelt** definieren.
- Der DbContext erbt von IdentityDbContext, **base.OnModelCreating zuerst**.
- Fehler kommen als **IdentityResult**, bei uns als ValidationProblem (400).
- Rollen liegen in **eigenen Tabellen**, gesetzt mit AddToRoleAsync.
- **401** nicht angemeldet, **403** keine Berechtigung, **429** zu viele Fehlversuche.
- Alte Migrationen **nie verändern**, neue Migration anlegen.